

超低时延密码杂凑算法LLH设计规范

冯秀涛、曾祥勇、王张宜、徐圣源、向泽军、王焕东、
唐明、唐灯、秦波、高博、张卫国、陈思维

2026. 06. 25

目 录

1. 算法概述.....	3
1.1 简介.....	3
1.2 符号.....	3
1.3 总体架构.....	3
1.4 算法参数与实例.....	4
1.4.1 算法参数.....	4
1.4.2 推荐实例配置.....	4
2. 算法详细设计.....	5
2.1 算法整体结构.....	5
2.2 工作流程.....	5
2.2.1 消息填充.....	5
2.2.2 吸收与挤压.....	6
2.3 置换函数 P	6
2.3.1 内部状态组织.....	6
2.3.2 置换函数 P 框架.....	7
2.4 轮函数ROUNDFUN.....	8
2.4.1 列替换.....	8
2.4.2 行移位.....	9
2.4.3 常数加.....	9
附录A 不同 w 取值及对应建议轮数下的完整轮常数参考值	11

1. 算法概述

1.1 简介

LLH是一族基于海绵结构变体设计的超低时延密码杂凑算法，支持比特长度不超过 2^{64} -1的输入消息以及256、512、768、1024等4种不同比特长度的输出杂凑值。该算法能够为 d 比特输出杂凑值提供 $d/2$ 比特的抗碰撞安全强度和 d 比特的原像安全强度，可适用于数字签名、消息认证码(MAC)生成、密钥派生(KDF)以及伪随机数生成等多种密码学应用场景。

LLH算法的核心是一个置换函数 P ，该函数基于切片技术设计，采用密码学性质良好的超低时延密码组件，通过对内部状态进行多轮迭代，在提供高性能的同时确保输出杂凑值的随机性与不可预测性。本规范主要定义了LLH密码杂凑算法的结构、处理流程及参数要求。

1.2 符号

本文用到的符号如下表1-1所示

表1-1 符号说明

$\oplus$	按位异或 (XOR) 运算
$\wedge$	按位与 (AND) 运算
$\lll$	在 w 比特宽度内的循环左移
$\ggg$	在 w 比特宽度内的循环右移
$\parallel$	比特串连接
0^k	长度为 k 的全零比特串
$S_{i:j}$	状态 S 中从第 i 比特到第 j 比特的字
σ_i	第 i 个字的移位偏移量
W_i	内部状态第 i 个字, $i=0,1,...,23$
RC_t	第 t 轮的轮常数，其宽度为 w 比特

1.3 总体架构

LLH采用海绵结构变体设计。该结构维护一个大小为 b 比特的内部状态 S ，状态被划分为两部分：

- 比特率：记为 r 比特，用于与输入消息块进行交互。

- 容量：记为 c 比特，用于保障算法的安全性，不直接与输入消息交互。

它们满足关系式： $b = r + c$ 。

算法处理过程分为两个阶段：

- (1) 吸收阶段：将填充后的输入消息分块，依次与状态的前 r 比特进行异或操作，每异或一次执行一次置换函数 P 。
- (2) 挤压阶段：执行置换函数 P ，从执行结果抽取前 r 比特作为输出杂凑值。

1.4 算法参数与实例

1.4.1 算法参数

LLH算法的具体安全强度由字长 w 及轮数 N_r 决定。内部状态 S 被组织为 $24 \times w$ 的比特矩阵，即总状态比特宽度 $b = 24 \times w$ 。其主要参数定义如下表1-2所示：

表1-2 算法参数说明

参数符号	名称	说明
w	字宽	内部状态行的比特宽度
N_r	轮数	置换函数 P 的迭代次数
d	输出长度	最终杂凑值的比特长度

1.4.2 推荐实例配置

LLH算法支持多种字宽配置，以适应不同的性能与安全需求。字宽 w 的标准取值为 32、64 、96或 128 比特。对应的内部状态大小如下表1-3所示：

表1-3 推荐算法参数配置

实例名称 (Variant)	字宽 w	内部状态 大小 b	典型输出 长度 d	比特率 (Rate)	容量 (Capacity)	轮数 N_r
LLH-256	32	768	256	256	512	16
LLH-512	64	1536	512	512	1024	24

LLH-768	96	2304	768	768	1536	32
LLH-1024	128	3072	1024	1024	2048	40

注：实例名称中的数字（如 256、512、768、1024）表示该配置下推荐的杂凑值输出长度。具体的安全强度取决于容量 c 的配置，通常碰撞抗性安全强度为 $\min(d/2, c/2)$ 比特。轮数的设计原则为 $8 + \lceil w/4 \rceil$ ，确保足够的混淆和扩散。

2. 算法详细设计

2.1 算法整体结构

LLH算法整体结构如图2-1所示。消息 m 经过填充后进入吸收阶段，每次通过置换函数 P 吸收一个消息块；随后进入挤压阶段，通过调用置换函数 P 生成输出杂凑值。

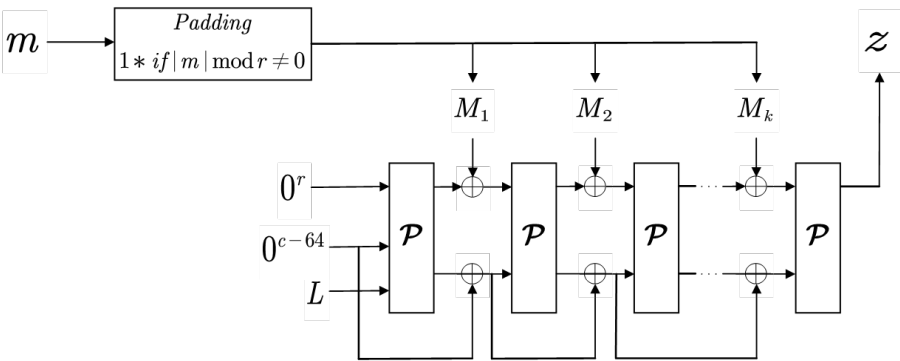


图2-1：LLH杂凑算法结构图

2.2 工作流程

2.2.1 消息填充

输入消息 M 的比特长度记为 L 。在进行吸收之前，需对 M 进行填充使其长度为比特率 r 的整数倍。填充规则如下：

- (1) 若 L 是 r 的整数倍（即 $L \bmod r = 0$ ）：不进行任何填充，直接将原消息作为填充后的消息 M' 。
- (2) 若 L 不是 r 的整数倍：在消息末尾添加若干个比特 '1'，直到长度达到 r 的最小整数倍。填充比特 '1' 的数量 $n = r - (L \bmod r)$ 。

消息在注入内部状态前按 little-endian 方式编码为比特串。具体地，同一 w 比特字内

按低地址到高地址的字节顺序由低位向高位填入；若消息末尾不足 1 字节，则有效比特先占低位，其余高位按填充规则补 `1`。例如，当 $w = 32$ 且输入字节序列为 `0A 01 03 04` 时，其注入对应的 32 比特字为 **0x0403010A**；若输入为 2 比特的消息 `10`，则填充 `1` 后的首字节为 **0xFD**。记填充后的消息序列为 $M' = M_1, M_2, \dots, M_k$ ，其中每个块 M_i 的长度为 r 比特。

2.2.2 吸收与挤压

设状态宽度为 b 比特 ($b = r + c$)，其中 r 为比特率， c 为容量。

(1) 初始化

初始状态 $S^{(0)} = 0^{b-64} \parallel L$ ，即将消息的原始长度 L 编码并嵌入到开始状态的最后 64 比特，其余比特保持为 0。

(2) 吸收

对于每个消息块 M_i ($1 \leq i \leq k$)，依次执行：

$$S^{(i)} = P(S^{(i-1)}) \oplus (M_i \parallel S_{r:b-1}^{(i-1)}),$$

(3) 挤压 (Squeezing)

计算

$$S^{(k+1)} = P(S^{(k)}),$$

则 $Z = S_{0:r-1}^{(k+1)}$ 为最终杂凑值。

2.3 置换函数 P

2.3.1 内部状态组织

置换函数 P 的输入为长度为 b 比特的内部状态 S 。状态 S 在逻辑上被组织为一个 $24 \times w$ 的二维比特矩阵，其中 w 为字长参数：

$$S = \begin{pmatrix} s_{0,0} & s_{0,1} & \cdots & s_{0,w-1} \\ s_{1,0} & s_{1,1} & \cdots & s_{1,w-1} \\ \vdots & \vdots & \ddots & \vdots \\ s_{23,0} & s_{23,1} & \cdots & s_{23,w-1} \end{pmatrix},$$

其中：

$s_{i,j}$ 表示第 i 行、第 j 列的比特, $0 \leq i < 24$, $0 \leq j < w$;

行: 每行包含 w 个比特, 称为一个“字”, 共24个字, 记为 $W_0, W_1, \dots, W_{23}$;

列: 每列包含 24 个比特, 称为一个“切片”。共有 w 个切片, 记为 $C_0, C_1, \dots, C_{w-1}$;

状态大小: 根据实例配置不同, 状态总大小 b 可为768、1536、2304或3072比特 (对应 w 分别为32、64、96、128)。

在吸收阶段, 输入消息块与状态的前 r 比特 (即前 $\lfloor r/w \rfloor$ 个字) 进行异或; 在挤压阶段, 杂凑值输出从状态的前 r 比特生成。

2.3.2 置换函数 P 框架

置换函数 P 基于切片技术设计。该函数接收 $24 \times w$ 比特的状态作为输入, 输出同等长度的状态。置换函数 P 由 N_r 轮轮函数 RoundFun 组成, 每轮轮函数 RoundFun 分别包含列替换、行移位及常数加三个步骤, 其流程经过如下图2-2所示:

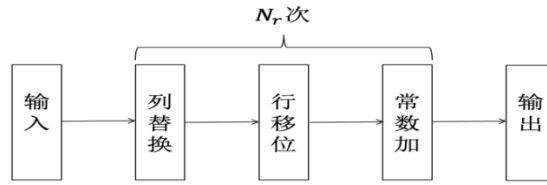


图2-2：置换 P 示意图

设 $X^{(t)}$ 为第 t 轮迭代后的状态, 则置换过程可表示为:

$$X^{(0)} = X_{input},$$

$$X^{(t+1)} = \text{RoundFun}(X^{(t)}), \quad 0 \leq t < N_r,$$

$$Y_{output} = X^{(N_r)}.$$

每一轮轮函数 RoundFun ($0 \leq t < N_r$) 由3个变换组成, 依次为:

- (1) 列替换: 对内部状态按照列进行非线性替换。
- (2) 行移位: 基于 w 比特字的循环移位。
- (3) 常数加: 异或特定常数, 消除对称性。

2.4 轮函数RoundFun

2.4.1 列替换

列替换是轮函数RoundFun中唯一的非线性变换操作。该操作对内部状态的 w 个切片并行处理，每个切片 C_j ($0 \leq j < w$)经过一个24比特超级S盒 $\mathcal{S}_{24}$ 变换。

2.4.1.1 超级S盒 $\mathcal{S}_{24}$

超级S盒 $\mathcal{S}_{24}$ 是一个24比特输入到24比特输出的变换。记24比特输入向量为 $(v_0, v_1, \dots, v_{23})$ 。

- 首先，将该向量编码为一个 6×4 的比特矩阵 $M_{6 \times 4}$ ，编码规则为按行优先填充：

$$M_{6 \times 4} = \begin{pmatrix} v_0 & v_1 & v_2 & v_3 \\ v_4 & v_5 & v_6 & v_7 \\ v_8 & v_9 & v_{10} & v_{11} \\ v_{12} & v_{13} & v_{14} & v_{15} \\ v_{16} & v_{17} & v_{18} & v_{19} \\ v_{20} & v_{21} & v_{22} & v_{23} \end{pmatrix}.$$

- 其次，对矩阵 $M_{6 \times 4}$ 的每一列（共 4 列，每列 6 比特）分别作用一个6比特S盒 $\mathcal{S}_6$ 变换，其定义如下：

$$\mathcal{S}_6(x) = x \oplus (x \ggg 2) \oplus ((x \ggg 1) \wedge (x \ggg 3)) \oplus 0x1B.$$

- 再次，对列操作后的结果进行错位重排，重排方式为：

表2-3 错位排列示意图

0	1	2	3	错位排列 →	4	9	18	23
4	5	6	7		8	1	22	15
8	9	10	11		12	17	6	3
12	13	14	15		20	5	10	19
16	17	18	19		0	21	14	11
20	21	22	23		16	13	2	7

- 最后，将错位排列后的矩阵每一行视作一个4比特行向量 R_i ($0 \leq i < 6$)，右乘一个 4×4 的矩阵 D 。设变换后的行向量 R'_i ，则有：

$$R'_i = R_i \times D \pmod{2},$$

其中矩阵 D 定义如下：

$$D = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}.$$

2.4.2 行移位

在列替换完成后，对内部状态的 24 个字 W_i （行）分别进行循环移位操作。设第 i 个字 W_i ($0 \leq i < 24$) 循环左移 σ_i 位，则

$$W_i = W_i \lll \sigma_i,$$

其中 σ_i 对不同字宽 w 的取值见表2-4。

表2-4 行循环移位 σ_i 具体取值

i	$w=32$	$w=64$	$w=96$	$w=128$
0	3	2	2	1
1	16	25	11	16
2	13	20	28	55
3	26	51	53	118
4	8	15	19	10
5	29	40	34	29
6	2	37	57	72
7	23	6	88	11
8	21	36	52	35
9	18	63	73	58
10	31	62	6	105
11	28	33	43	48
12	10	1	5	76
13	15	30	32	103
14	4	31	67	26
15	9	4	14	101
16	7	38	70	5
17	20	5	7	36
18	17	8	48	91
19	30	47	1	42
20	12	19	55	78
21	1	52	94	113
22	6	57	45	44
23	27	34	4	127

2.4.3 常数加

该操作仅针对内部状态的最后一个字 W_{23} 异或一个轮常数 RC_t ，即

$$W_{23} \leftarrow W_{23} \oplus RC_t,$$

其中 RC_t 为第 t 轮的 w 比特轮常数($0 \leq t < N_r$)。

轮常数序列 $\{RC_0, RC_1, \dots, RC_{N_r-1}\}$ 由自然常数 e (Euler's number, $e \approx 2.71828 \dots$) 确定，具体生成步骤如下：

1. 取小数部分：取自然常数 e 的小数部分 ($e - 2$)；
2. 二进制转换：将该小数部分转换为二进制比特流；
3. 分段选取：从比特流的第 0 位开始，每轮依次截取 w 比特。即第一轮 ($t = 0$) 选取第 $0 \sim w - 1$ 位，第二轮 ($t = 1$) 选取第 $w \sim 2w - 1$ 位，依此类推。

在附录A中列出了基于上述规则生成的对于 w 不同取值时的常数参考值。

附录A 不同 w 取值及对应建议轮数下的完整轮常数参考值

表A-1 $w = 32$ 时0~15完整轮常数参考值

轮数 t	轮常数取值 RC
0	$RC_0 = 0xB7E15162$
1	$RC_1 = 0x8AED2A6A$
2	$RC_2 = 0xBF715880$
3	$RC_3 = 0x9CF4F3C7$
4	$RC_4 = 0x62E7160F$
5	$RC_5 = 0x38B4DA56$
6	$RC_6 = 0xA784D904$
7	$RC_7 = 0x5190CFEF$
8	$RC_8 = 0x324E7738$
9	$RC_9 = 0x926CFBE5$
10	$RC_{10} = 0xF4BF8D8D$
11	$RC_{11} = 0x8C31D763$
12	$RC_{12} = 0xDA06C80A$
13	$RC_{13} = 0xBB1185EB$
14	$RC_{14} = 0x4F7C7B57$
15	$RC_{15} = 0x57F59584$

表A-2 $w = 64$ 时0~23完整轮常数参考值

轮数 t	轮常数取值 RC
0	$RC_0 = 0xB7E151628AED2A6A$
1	$RC_1 = 0xBF7158809CF4F3C7$
2	$RC_2 = 0x62E7160F38B4DA56$
3	$RC_3 = 0xA784D9045190CFEF$
4	$RC_4 = 0x324E7738926CFBE5$
5	$RC_5 = 0xF4BF8D8D8C31D763$

6	$RC_6 = 0xDA06C80ABB1185EB$
7	$RC_7 = 0x4F7C7B5757F59584$
8	$RC_8 = 0x90CFD47D7C19BB42$
9	$RC_9 = 0x158D9554F7B46BCE$
10	$RC_{10} = 0xD55C4D79FD5F24D6$
11	$RC_{11} = 0x613C31C3839A2DDF$
12	$RC_{12} = 0x8A9A276BCFBFA1C8$
13	$RC_{13} = 0x77C56284DAB79CD4$
14	$RC_{14} = 0xC2B3293D20E9E5EA$
15	$RC_{15} = 0xF02AC60ACC93ED87$
16	$RC_{16} = 0x4422A52ECB238FEE$
17	$RC_{17} = 0xE5AB6ADD835FD1A0$
18	$RC_{18} = 0x753D0A8F78E537D2$
19	$RC_{19} = 0xB95BB79D8DCAEC64$
20	$RC_{20} = 0x2C1E9F23B829B5C2$
21	$RC_{21} = 0x780BF38737DF8BB3$
22	$RC_{22} = 0x00D01334A0D0BD86$
23	$RC_{23} = 0x45CBFA73A6160FFE$

表A-3 $w = 96$ 时0~31完整轮常数参考值

轮数 t	轮常数取值 RC
0	$RC_0 = 0xB7E151628AED2A6ABF715880$
1	$RC_1 = 0x9CF4F3C762E7160F38B4DA56$
2	$RC_2 = 0xA784D9045190CFEF324E7738$
3	$RC_3 = 0x926CFBE5F4BF8D8D8C31D763$
4	$RC_4 = 0xDA06C80ABB1185EB4F7C7B57$
5	$RC_5 = 0x57F5958490CFD47D7C19BB42$
6	$RC_6 = 0x158D9554F7B46BCED55C4D79$
7	$RC_7 = 0xFD5F24D6613C31C3839A2DDF$

8	$RC_8 = 0x8A9A276BCFBFA1C877C56284$
9	$RC_9 = 0xDAB79CD4C2B3293D20E9E5EA$
10	$RC_{10} = 0xF02AC60ACC93ED874422A52E$
11	$RC_{11} = 0xCB238FEEE5AB6ADD835FD1A0$
12	$RC_{12} = 0x753D0A8F78E537D2B95BB79D$
13	$RC_{13} = 0x8DCAEC642C1E9F23B829B5C2$
14	$RC_{14} = 0x780BF38737DF8BB300D01334$
15	$RC_{15} = 0xA0D0BD8645CBFA73A6160FFE$
16	$RC_{16} = 0x393C48CBBBCA060F0FF8EC6D$
17	$RC_{17} = 0x31BEB5CCEED7F2F0BB088017$
18	$RC_{18} = 0x163BC60DF45A0ECB1BCD289B$
19	$RC_{19} = 0x06CBBFEA21AD08E1847F3F73$
20	$RC_{20} = 0x78D56CED94640D6EF0D3D37B$
21	$RC_{21} = 0xE67008E186D1BF275B9B241D$
22	$RC_{22} = 0xEB64749A47DFDFB96632C3EB$
23	$RC_{23} = 0x061B6472BBF84C26144E49C2$
24	$RC_{24} = 0xD04C324EF10DE513D3F5114B$
25	$RC_{25} = 0x8B5D374D93CB8879C7D52FFD$
26	$RC_{26} = 0x72BA0AAE7277DA7BA1B4AF14$
27	$RC_{27} = 0x88D8E836AF14865E6C37AB68$
28	$RC_{28} = 0x76FE690B571121382AF341AF$
29	$RC_{29} = 0xE94F77BCF06C83B8FF5675F0$
30	$RC_{30} = 0x979074AD9A787BC5B9BD4B0C$
31	$RC_{31} = 0x5937D3EDE4C3A79396215EDA$

表A-4 $w = 128$ 时0~39完整轮常数参考值

轮数 t	轮常数取值 RC
0	$RC_0 = 0xB7E151628AED2A6ABF7158809CF4F3C7$
1	$RC_1 = 0x62E7160F38B4DA56A784D9045190CFEF$

2	$RC_2 = 0x324E7738926CFBE5F4BF8D8D8C31D763$
3	$RC_3 = 0xDA06C80ABB1185EB4F7C7B5757F59584$
4	$RC_4 = 0x90CFD47D7C19BB42158D9554F7B46BCE$
5	$RC_5 = 0xD55C4D79FD5F24D6613C31C3839A2DDF$
6	$RC_6 = 0x8A9A276BCFBFA1C877C56284DAB79CD4$
7	$RC_7 = 0xC2B3293D20E9E5EAF02AC60ACC93ED87$
8	$RC_8 = 0x4422A52ECB238FEEE5AB6ADD835FD1A0$
9	$RC_9 = 0x753D0A8F78E537D2B95BB79D8DCAEC64$
10	$RC_{10} = 0x2C1E9F23B829B5C2780BF38737DF8BB3$
11	$RC_{11} = 0x00D01334A0D0BD8645CBFA73A6160FFE$
12	$RC_{12} = 0x393C48CBBBCA060F0FF8EC6D31BEB5CC$
13	$RC_{13} = 0xEED7F2F0BB088017163BC60DF45A0ECB$
14	$RC_{14} = 0x1BCD289B06CBBFEA21AD08E1847F3F73$
15	$RC_{15} = 0x78D56CED94640D6EF0D3D37BE67008E1$
16	$RC_{16} = 0x86D1BF275B9B241DEB64749A47DFDFB9$
17	$RC_{17} = 0x6632C3EB061B6472BBF84C26144E49C2$
18	$RC_{18} = 0xD04C324EF10DE513D3F5114B8B5D374D$
19	$RC_{19} = 0x93CB8879C7D52FFD72BA0AAE7277DA7B$
20	$RC_{20} = 0xA1B4AF1488D8E836AF14865E6C37AB68$
21	$RC_{21} = 0x76FE690B571121382AF341AFE94F77BC$
22	$RC_{22} = 0xF06C83B8FF5675F0979074AD9A787BC5$
23	$RC_{23} = 0xB9BD4B0C5937D3EDE4C3A79396215EDA$
24	$RC_{24} = 0xB1F57D0B5A7DB461DD8F3C75540D0012$
25	$RC_{25} = 0x1FD56E95F8C731E9C4D7221BBED0C62B$
26	$RC_{26} = 0xB5A87804B679A0CAA41D802A4604C311$
27	$RC_{27} = 0xB71DE3E5C6B400E024A6668CCF2E2DE8$
28	$RC_{28} = 0x6876E4F5C50000F0A93B3AA7E6342B30$
29	$RC_{29} = 0x2A0A47373B25F73E3B26D569FE2291AD$
30	$RC_{30} = 0x36D6A147D1060B871A2801F978376408$

31	$RC_{31} = 0x2FF592D9140DB1E9399DF4B0E14CA8E8$
32	$RC_{32} = 0x8EE9110B2BD4FA98EED150CA6DD89322$
33	$RC_{33} = 0x45EF7592C703F532CE3A30CD31C070EB$
34	$RC_{34} = 0x36B4195FF33FB1C66C7D70F93918107C$
35	$RC_{35} = 0xE2051FED33F6D1DE9491C7DEA6A5A442$
36	$RC_{36} = 0xE154C8BB6D8D0362803BC248D414478C$
37	$RC_{37} = 0x2AFB07FFE78E89B9FECA7E3060C08F0D$
38	$RC_{38} = 0x61F8E36801DF66D1D8F9392E52CAEF06$
39	$RC_{39} = 0x53199479DF2BE64BBAAB008CA8A06FDA$